



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unitat 06 — Arrays

Bloc 2

1r CFGS DAW · Programació

Sergi García Barea



En aquesta sessió

El funcionament intern, els objectes de veritat i el gimnàs d'examen

- 1 Arrays i mètodes: passar i tornar arrays (la **referència**).
- 2 Aplicacions: `String`, `char[]`, **objectes** i taules de dades.
- 3 **Be the Code**: invertir sense array auxiliar i buscar totes les posicions.
- 4 Els **6 monstres** dels arrays i com depurar-los.
- 5 **Repàs**: l'aparcament a examen.

 Necessites el Bloc 1: índexs, `length`, `for` i `for-each`, matrius i la classe `Arrays`.

Arrays i mètodes: passant el testimoni

ArraysMetodos.java

99?

```
public class ArraysMetodos {  
    public static void main(String[] args) {  
        int[] edades = {10, 20, 30};  
        modificar(edades);  
        System.out.println(edades[0]);  
    }  
    public static void modificar(int[] arr) {  
        arr[0] = 99;  
    }  
}
```

Imprimix **99**, no 10: el mètode ha canviat la plaça 0 d'un array creat en `main`.

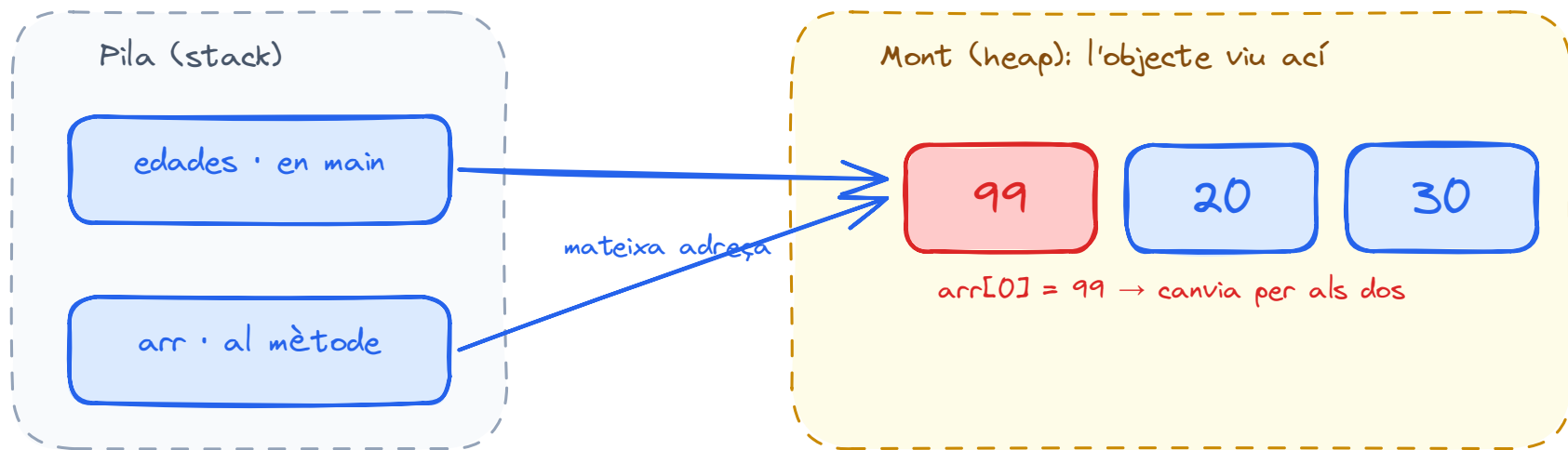
Com és possible si Java passa els arguments **per valor**?



Perquè es passa per valor la **còpia de la referència**: l'array no es copia, només l'adreça. La còpia i l'original apunten al mateix aparcament.

La referència, dibuixada

```
modificar(edades);    →    arr[0] = 99;
```



⚠ ⚠ **Açò NO és pas per referència de veritat:** Java mai passa la variable per referència. es copia la REFERÈNCIA (l'adreça), mai l'array → un sol objecte, dos noms
Passa una còpia de la referència — per això es diu *pass-by-value*.
per això el canvi dins del mètode es veu fora: main imprimix 99

An error occurred on this slide. Check the terminal for more information.

Reassignar no és modificar



`arr = otroArray;` dins del mètode

Només reassignes la teua **còpia de la referència**: l'original segueix intacte.



`arr[i] = ...;` dins del mètode

Toca el **contingut** de l'objecte compartit: el canvi es veu fora.



Per a modificar l'original, toca **elements** (o atributs de l'objecte), mai la variable. A dues variables que apunten al mateix objecte se'ls diu **àlies**.

Mètodes que treballen amb arrays

Tornar un array nou

Rebre un array per a calcular

Media.java

```
public static double media(int[] notas) {  
    int suma = 0;  
    for (int n : notas) {  
        suma += n;  
    }  
    return (double) suma / notas.length;  
}
```

- 1 duplicar(int[] arr) amb arr[i] *=
- 2 : modifica **sense return**.

Cuadrados.java

```
public static int[] cuadrados(int n) {  
    int[] resultado = new int[n];  
    for (int i = 0; i < n; i++) {  
        resultado[i] = (i + 1) * (i + 1);  
    }  
    return resultado;  
}  
  
// cuadrados(4) → [1, 4, 9, 16]
```

i Tornar un array = tornar una **referència** a un objecte del heap. Si no vols que te'l toquen, torna una còpia (`Arrays.copyOf`).

El famós main(String[] args)

Des de la U02 escriviu `main(String[] args)` sense parar-vos a pensar. És un mètode que rep un **array de Strings**: els arguments de la línia de comandes.

```
> java Saludo Ana
Hola, Ana
> // args = {"Ana"} · args.length = 1
```

⚠ Executar **sense arguments** i accedir a `args[0]` → `ArrayIndexOutOfBoundsException` — el mateix error del Bloc 1, ara amb la cara de `args`.

✓ Comprova sempre abans d'accedir: `if (args.length > 0) { ... }`



Be the Code: l'array al quadrat

BeTheArrayRevelde.java

```
public class BeTheArrayRevelde {  
    public static void main(String[] args) {  
        int[] nums = {1, 2, 3, 4, 5};  
        for (int i = 0; i < nums.length; i++) {  
            nums[i] = nums[i] * nums[i];  
        }  
        System.out.println(nums[2]);  
    }  
}
```

Què imprimix?

a) 3 · b) 6 · c) 9 · d) 25

El c. Es fa el quadrat de cada número al lloc: {1, 4, 9, 16, 25} → `nums[2] = 9`. El patró `arr[i] = ...` és el mateix que usaries per a modificar un array rebut en un mètode.

Comprovació ràpida

? `int[] a = {1,2,3}; cambiar(a);` amb `cambiar(int[] x) { x[0] = 7; }` — quant val `a[0]` ?

? I si el mètode fa `x = new int[]{9,9,9}` en comptes de tocar `x[0]` ?

? `String` es comporta com un array o com un primitiu al passar-lo a un mètode?

1. 7: el mètode modifica el contingut de l'objecte compartit. · **2.** Res: l'original segueix `{1, 2, 3}` ; només reassigna la còpia local de la referència. · **3.** La referència es copia, però `String` és immutable: cap mètode no pot canviar el seu contingut.

An error occurred on this slide. Check the terminal for more information.

char[] vs String: semblants però no iguals

Cosa	String	char[]
Immutable?	Sí: no es pot canviar	No: pots tocar cada plaça
Mètodes	<code>length()</code> , <code>charAt()</code> , <code>substring()</code> ...	No en té (uses bucles)
Canviar una lletra?	No (en crees una altra)	Sí: <code>vocales[0] = 'A';</code>
Passar a mètode	Es comporta com immutable	Es compartix com qualsevol objecte



Si necessites «canviar una lletra»: passa la `String` a `char[]`, modifica-la i torna a construir la `String`. Esta idea tornarà en la unitat de fitxers i regex.

Arrays d'objectes: l'aparcament de persones

Alumno.java

```
class Alumno {  
    String nombre;  
    int nota;  
    Alumno(String nombre, int nota) {  
        this.nombre = nombre;  
        this.nota = nota;  
    }  
}  
  
// al main:  
Alumno[] alumnos = new Alumno[3];  
alumnos[0] = new Alumno("Ana", 8);  
alumnos[1] = new Alumno("Bruno", 5);  
alumnos[2] = new Alumno("Carla", 10);
```

⚠ Un `Alumno[]` acabat de crear està ple de **null**, no d'alumnes. Accedir a `alumnos[0].nombre` sense crear l'objecte → `NullPointerException`. Primer `new Alumno(...)`, després usar.

I el bucle? **Idèntic** al dels números:

Comptar aprovats

```
int aprobados = 0;  
for (Alumno a : alumnos) {  
    if (a.nota ≥ 5) {  
        aprobados++;  
    }  
}
```

Taules de dades: double[][]

Notas.java

```
double[][] notas = {  
    {8.0, 7.5, 9.0}, // Ana  
    {5.0, 6.0, 4.5}, // Bruno  
    {9.5, 8.0, 10.0} // Carla  
};
```

Cada **fila** és un alumne; cada **columna**, una assignatura (Mat, Llengua, Anglès).

Mitjana per alumne

```
for (int i = 0; i < notas.length; i++) {  
    double suma = 0;  
    for (int j = 0; j < notas[i].length; j++) {  
        suma += notas[i][j];  
    }  
    System.out.println("Alumne " + i  
        + ": mitjana "  
        + (suma / notas[i].length));  
}
```



Els programes que «porten el compte» de la vida real són, en el fons, açò.



Be the Code: la cerca de la pel·lícula

BeTheCatalogo.java

```
public class BeTheCatalogo {  
    public static void main(String[] args) {  
        String[] peliculas = {"Alien",  
                               "Matrix", "Gladiator",  
                               "Matrix", "Coco"};  
        String buscada = "Matrix";  
        int cuantas = 0;  
        for (String p : peliculas) {  
            if (p.equals(buscada)) {  
                cuantas++;  
            }  
        }  
        System.out.println(cuantas);  
    }  
}
```

Què imprimix?

a) 1 · b) 2 · c) 3 · d) "Matrix"

L'b. «Matrix» apareix en les posicions 1 i 3:
dues voltes. I fixa't en el `.equals()` :
amb `=` compararies referències i no
trobaria cap.

Comprovació ràpida

?

Què hi ha a les places d'un `Alumno[]` acabat de crear?

?

Per què `fruta = "pera"` és un error amb `String`?

?

Pots modificar una lletra d'una `String` ? I d'un `char[]` ?

1. `null` : cal crear els objectes amb `new Alumno(...)` . **2.** `=` compara referències (és el mateix objecte?), no el contingut: amb `String` cal `.equals()` . **3.** No: `String` és immutable. Sí: `char[]` no ho és (`vocales[0] = 'A'` funciona).



Be the Code: el gimnàs

Este punt no té teoria nova: té **tres reptes**. Quan et demanen «inverteix este array» en una entrevista, ningú et deixa usar `Arrays`: cal saber-ho fer a mà.

- 1 Dos punters: `izquierda = 0` i `derecha = array.length - 1`.
- 2 Mentre `izquierda < derecha`: intercanvia `array[izquierda]` i `array[derecha]` amb una variable `temp`.
- 3 `izquierda++` i `derecha--`: els punters s'apropen al centre.
- 4 Quan es creuen, ja està: tot l'array invertit al lloc.



REPTE 1: invertir l'array al lloc

RetoInverso.java

```
public class RetoInverso {  
    public static void invertir(  
        int[] array) {  
        // 🧠 EL TEU CODI ACÍ  
    }  
    public static void main(String[] args) {  
        int[] datos = {1, 2, 3, 4, 5};  
        invertir(datos);  
        for (int n : datos) {  
            System.out.print(n + " ");  
        }  
    }  
}
```

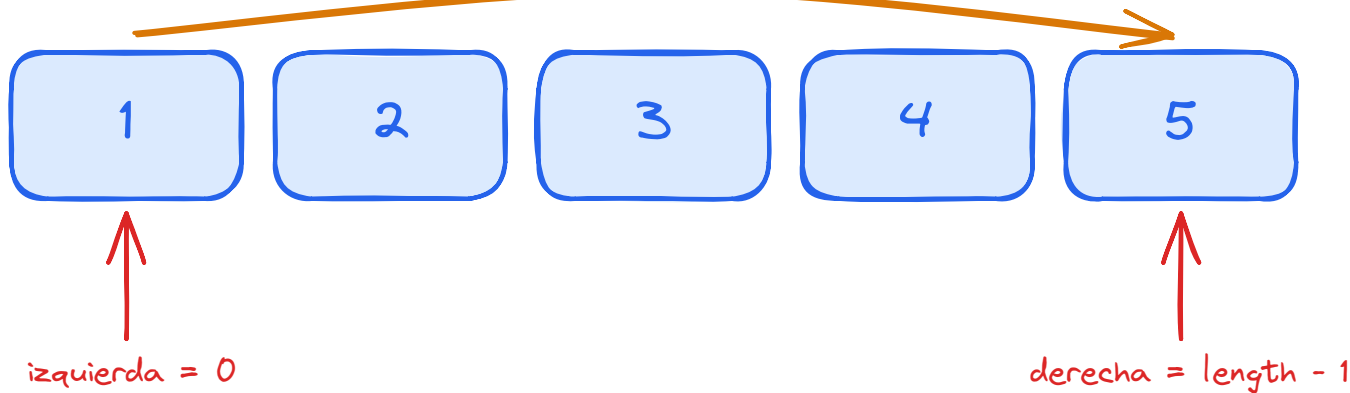
Objectiu: mostrar `5 4 3 2 1` sense crear un altre array.

- 1 Declara els dos punters.
- 2 Escriu el `while` amb la condició correcta.
- 3 Intercanvia els dos elements amb una variable temporal.
- 4 Mou els punters cap al centre.

El truc dels dos punters, dibuixat

```
while (izquierda < derecha) { intercanvia; }
```

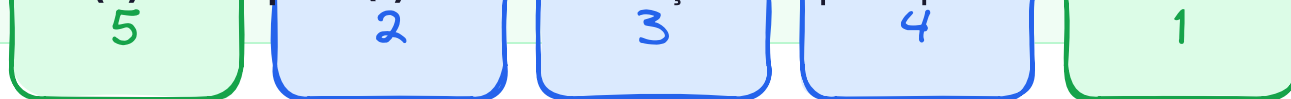
intercanvia amb una variable temp



cada volta: intercanvia · izquierda++ · derecha--



Al lloc: $O(n)$ de temps i $O(1)$ de memòria. Açò és el que impressiona en una entrevista.



després del primer intercanvi (els extrems, ja al seu lloc)

quan es creuen els punters, ja està: 5 4 3 2 1 · sense cap array auxiliar



Solució del repte 1

● ● ● RetoInverso.java — solució

```
public static void invertir(int[] array) {  
    int izquierda = 0;  
    int derecha = array.length - 1;  
    while (izquierda < derecha) {  
        int temp = array[izquierda];  
        array[izquierda] = array[derecha];  
        array[derecha] = temp;  
        izquierda++;  
        derecha--;  
    }  
}
```



L'error més comú: crear un array auxiliar quan no cal. Al lloc, amb dos punters i una variable temporal, és suficient — i molt més eficient.



REPTE 2: buscar totes les posicions

`binarySearch` et dona **una** posició. Este repte demana **totes**:

RetoBusqueda.java

```
public static int[] posiciones(  
    int[] datos, int buscado) {  
    // 🧠 EL TEU CODI ACÍ  
    return new int[0];  
}  
  
// datos = {3, 7, 2, 7, 9, 7, 1}  
// posiciones(datos, 7) → [1, 3, 5]  
// posiciones(datos, 5) → []
```

- 1 Primera passada: **compta** quantes voltes apareix.
- 2 Crea l'array de resultats amb **eixa grandària**.
- 3 Segona passada: **ompli** les posicions.



Quan la grandària depèn de les dades, es fa en **dues passades**: no pots crear l'array fins a saber quantes places necessita.

✓ Solució del repte 2: el patró de les dues passades

Passada 1: comptar

```
public static int[] posiciones(  
    int[] datos, int buscado) {  
    int cuantas = 0;  
    for (int i = 0; i < datos.length; i++) {  
        if (datos[i] == buscado) {  
            cuantas++;  
        }  
    }  
}
```

Passada 2: crear i omplir

```
int[] resultado = new int[cuantas];  
int k = 0;  
for (int i = 0; i < datos.length; i++) {  
    if (datos[i] == buscado) {  
        resultado[k++] = i;  
    }  
}  
return resultado;  
}
```

✓ Un mètode, dos bucles: el primer compta, el segon ompli. Patró que repetiràs sovint.



EL LÍO: l'aparcament que es va rebel·lar

L'encarregat volia «deixar les places imparells buides». Alguna cosa fa mala olor:

ParkingLioso.java

```
public class ParkingLioso {  
    public static void vaciarImpares(  
        int[] arr) {  
        for (int i = 0; i < arr.length; i++) {  
            if (arr[i] % 2 == 1) {  
                arr[i] = null; // 🚗 compila açò?  
            }  
        }  
    }  
}
```

No compila. `arr` és un `int[]`, i `int` és un tipus primitiu: **no pot valer null**. `null` només cap en variables de tipus objecte (`String`, `Integer`, `Alumno` ...).

i Per a «buidar» un `int[]`, posa un **valor de sentinella** (0 o -1). I si necessites places buides de veritat: arrays d'objectes o col·leccions (U12).

🕒 Don Tip: pregunta sempre: *quin tipus de dada guarda este array?*



La galeria dels 6 monstres

Monstre	Síntoma	Cura
1. ArrayIndexOutOfBoundsException	Accedir fora de <code>0...length - 1</code>	Revisar límits i el <code><=</code> del bucle
2. NullPointerException	Tocar una plaça sense objecte	<code>if (arr[i] != null)</code> o crear els objectes
3. Imprimir sense <code>toString</code>	<code>[I@6d06d69c</code>	<code>Arrays.toString</code> (o <code>deepToString</code>)
4. Comparar amb <code>=</code> / <code>equals</code>	<code>false</code> amb contingut igual	<code>Arrays.equals(a, b)</code>
5. <code>length</code> / <code>length()</code> / <code>size()</code>	No compila	array <code>length</code> · String <code>length()</code> · col·lecció <code>size()</code>
6. Modificar amb <code>for-each</code>	L'array no canvia	for amb índex: <code>a[i] = ...</code>



El 90% dels «arrays rebel·les» s'arreglen llegint el missatge d'error amb calma.



CONRAD vs el món: l'array que no es calla



«Un for amb quina condició? — *Doncs $i \leq \text{numeros.length}$... ¿ $\leq$?!* L'últim índex vàlid és **length - 1**. $\leq$ et porta a la plaça fantasma i d'allí no torna ningú.»



«Pregunta: què tens a la plaça? — *No ho sé, la vaig crear amb `new String[10]`. I quantes places has omplit? — Doncs... bé... ¡AH! ¡CAP! Un `String[]` acabat de crear és una fila de **null**. Si no fiques objectes, no hi ha objectes.»*



«I el favorit de tots: *el meu bucle que duplica no funciona*. — Com el recorres? — *Amb `for-each`*. ¡EL `for-each` NO MODIFICA! És un robot lector de matrícules: llig, però no repinta. Per a repintar, índex i claudàtors.»



Depurar arrays sense perdre el cap

- 1 Imprimeix l'array complet** amb `Arrays.toString()` a cada pas: vore les dades arregla la mitat dels misteris.
- 2 Comprova els límits abans de tocar:** si vas a accedir a `a[i + 1]`, que `i` no arribi a `length - 1`.
- 3 Usa el depurador:** breakpoint al bucle i observa `i` a cada volta. Si es passa de `length - 1`, ho veus a la primera.
- 4 Paper i boli:** simula el bucle a mà amb un array de 3 elements. En ple segle XXI, i segueix funcionant.
- 5 Aïlla l'error:** primer ompli i comprova; després el bucle; després el càlcul. Si falla, sabràs quin tram és.

i El missatge d'excepció no és el teu enemic: és un detectiu que et diu la línia i el motiu.
`Index 5 out of bounds for length 5` no deixa lloc a dubtes. **LLEGEIX-LO** abans de tocar res.



Be the Code: el caçador de monstres

CazaMonstruos.java



```
public class CazaMonstruos {  
    public static void main(String[] args) {  
        int[] datos = new int[5];  
        for (int i = 1; i ≤ datos.length; i++) {  
            datos[i] = i * 10;  
        }  
        System.out.println(  
            java.util.Arrays.toString(datos));  
    }  
}
```

Què ocorre?

- a) Imprimix [10, 20, 30, 40, 50]
- b) Imprimix [0, 10, 20, 30, 40]
- c) ArrayIndexOutOfBoundsException
- d) NullPointerException

El c (monstre 1 en acció). Quan `i` val 5, `datos[5]` no existix: les places vàlides van de 0 a 4. I de passada, la plaça 0 es queda sense tocar (el bucle comença en 1): un altre ensurt per als despistats.

Repàs: qui soc?

1 Sóc un aparcament de grandària fixa: guarde moltes dades del mateix tipus sota un sol nom.

2 Sóc el número de cada plaça, i comence en 0, per a confusió general.

3 Em llancen quan demanen la plaça que no existix.

4 Sóc el bucle de només lectura: «per a cada X en Y».

5 Sóc la classe utilitària amb `toString`, `sort`, `binarySearch`, `copyOf` i `fill`.

6 Sóc l'atribut que diu quantes places hi ha, sense parèntesis.

1. L'array · 2. L'índex · 3. La `ArrayIndexOutOfBoundsException` · 4. El for-each · 5. La classe `Arrays` · 6. `length`

Repàs: el joc de les decisions

1 Últim índex vàlid de `int[] a = new int[8]` ? — a) 7 · b) 8 · c) 9

2 Valor de cada plaça de `new boolean[4]` ? — a) true · b) false · c) null

3 `binarySearch` sobre un array **desordenat**? — a) Excepció · b) Impredictible · c) Ordena primer

4 `println(a.length)` per a `new int[10]` ? — a) 9 · b) 10 · c) [10]

5 Bucle per a recórrer un array **cap arrere**? — a) for-each · b) for amb índex · c) Qualsevol

1. **a** — 7 (de 0 a length - 1) · 2. **b** — false · 3. **b** — impredictible: exigix ordre · 4. **b** — 10 · 5. **b** — for cap arrere

Repàs: preguntes d'entrevista de treball

1 «Explicam'ho, com si jo fóra la teua iaia: **què és un array?**»

2 «Inverteix este array **sense crear-ne un altre**. Ara digues-me quanta memòria extra necessites.»

3 «Quina és la diferència entre **length**, **length()** i **size()**?»

4 «Quan usaries **for-each** i quan un **for** amb índex?»

5 «Com compares dos arrays per a saber si tenen el **mateix contingut?**»

6 «Escriu el codi que torna la **nota més alta** d'un array.»



Preguntes reals per a un programador Java júnior. Si saps respondre les sis, la unitat està apresada.

No hi ha preguntes tontes 🙋

? Per què el primer índex és 0 i no 1?

Perquè l'índex és una **distància** des del principi, no un número de plaça. La primera casa és a 0 passes de tu. Contar des de 0 evita l'off-by-one en milers de càlculs.

? Un array pot canviar de grandària?

No. És **grandària fixa** per sempre. Quan necessites «més places», es crea un array nou i es copia (`Arrays.copyOf`). Per això existixen les col·leccions (U12).

? Puc usar `sort` i `binarySearch` en lloc dels algorismes a mà?

En programes reals, sí. Però en la U07 aprendràs **com funcionen per dins** (bombolla, cerca binària) — i en una entrevista, et demanaran l'algoritme a mà.

? Què guarda la variable `args` del `main`?

Un `String[]` amb els arguments de la línia de comandes: `args[0]` és el primer i `args.length`, quants n'hi ha.

Resum del bloc 2 en 5 frases

- 1 Els arrays es passen per **referència** (còpia de la referència): modificar-los dins d'un mètode es veu fora. Els primitius, per **valor**.
- 2 Els arrays guarden **el que siga**: String (usa `.equals()`), `char[]`, objectes (comencen en `null`) i taules 2D.
- 3 **Invertir al lloc**: dos punters, intercanvi amb `temp` i `while (izquierda < derecha)`.
- 4 Grandària dependent de les dades → patró de **dues passades**: comptar primer, crear i omplir després.
- 5 Els **6 monstres** tenen nom, missatge i cura — i el missatge d'excepció és una pista: llig-lo abans de tocar res.

